

Java Collection Framework

What is Collection ?

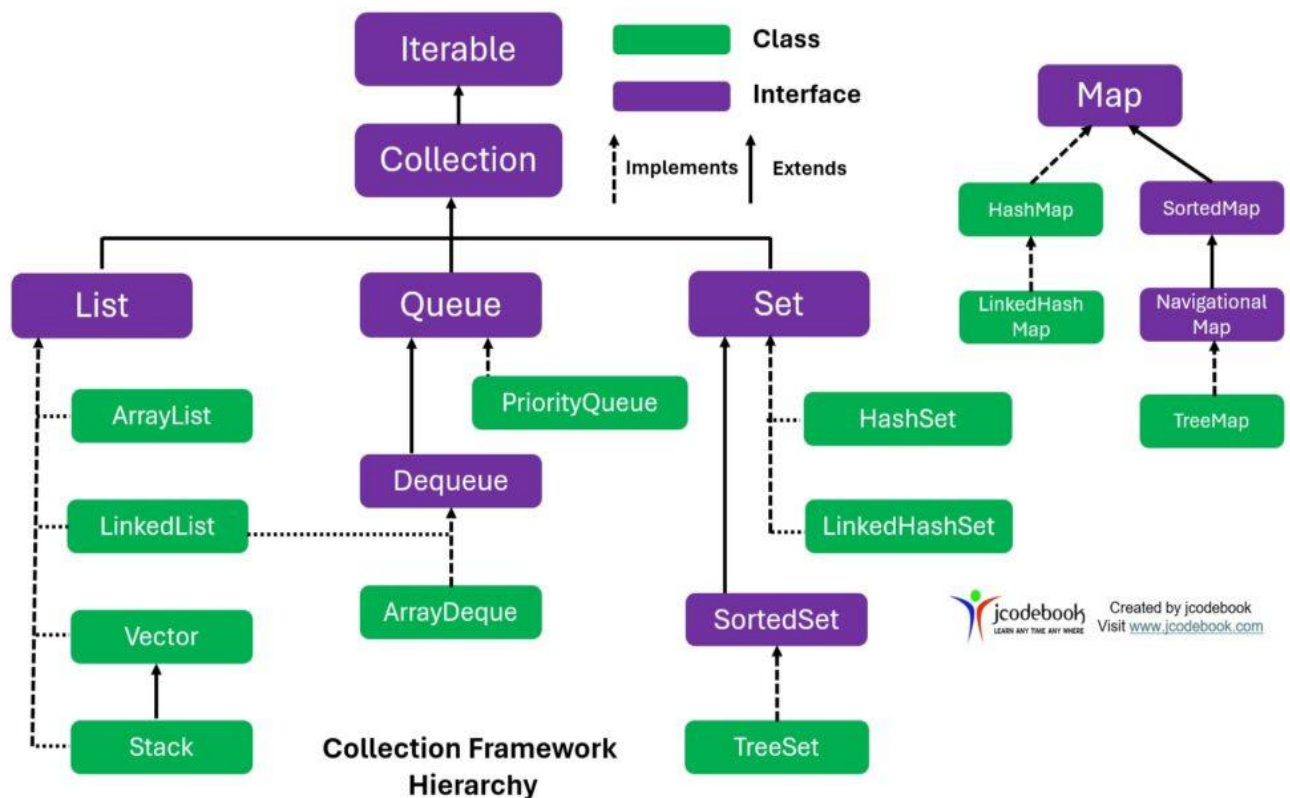
- A Collection is a single unit that groups multiple data elements together.
- Example: A list of employee names.

What is Java Collection Framework ?

- A set of classes and interfaces in Java.
- Used to store and manage groups of objects.
- Provides data structures like List, Set, Queue, and Map.
- Makes data handling easy and efficient.
- Simplifies storing, searching, and processing data.

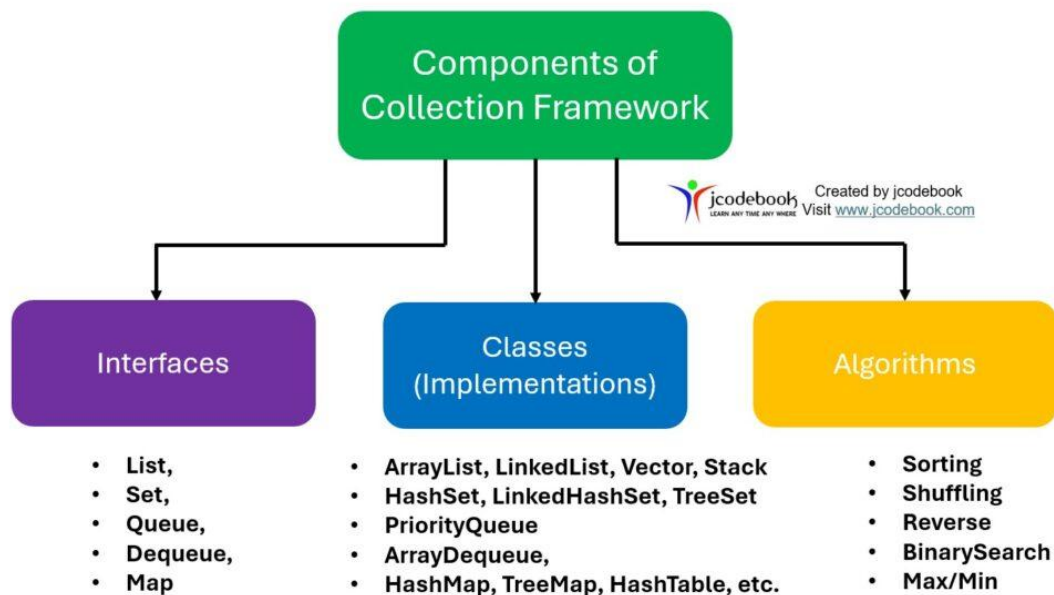
Hierarchy of Collection Framework in Java :-

- The Java Collection Framework has two root interfaces:
 1. Collection interface - `java.util.Collection`
 2. Map interface - `java.util.Map`



Components of the Collection Framework :-

The Java Collection Framework consists of three major components:



Components of Collection Framework in Java

1. **Interfaces:** Define the abstract data types for collections.
2. **Classes (Implementations):** Provide concrete implementations of the collection interfaces.
3. **Algorithms:** Provide static methods for performing common tasks such as sorting, searching, and modifying collections.

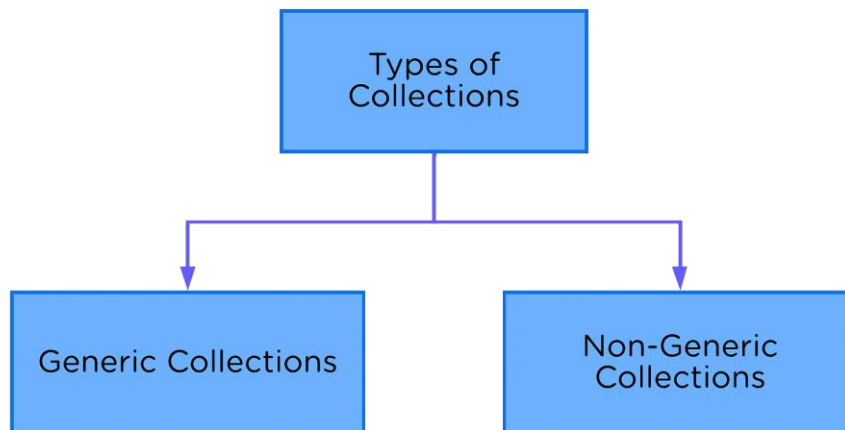
Needs for Java Collection Framework:

1. To store and manipulate data easily.
2. To provide reusability of code.
3. To provide performance and flexibility.
4. To ensure type safety (with Generics).
5. To maintain consistency across data structures.
6. To improve code readability and maintainability.
7. To support data manipulation algorithms.

Types of Collection :-

There are two types of collection are as follows:

- 1) Non-Generic Collections (Before Java 1.5)
- 2) Generic Collections (After Java 1.5)



1) Generic Collection :-

- Introduced in Java 1.5 for type safety.
- Can store only one type of data (homogeneous).
- No need for type casting.
- Errors detected at compile time.

2) Non-Generic Collection :-

- Can store different types of data (heterogeneous).
- No type safety.
- Type casting required while retrieving elements.
- Errors occur at runtime.

Advantages of Collection Framework :-

- Reduces programming effort.
- Increases performance..
- Provides ready-made data structures.
- Supports generic types.
- Easy sorting and searching utilities.

Syntax of Collection :-

List collectionName=new ArrayList();

OR

ArrayList collectionName=new ArrayList();

Methods of Collection Interface :-

No.	Method	Description
1	add()	Adds an element
2	addAll()	Adds all elements from another collection
3	remove()	Removes an element
4	removeAll()	Removes all matching elements
5	retainAll()	Keeps common elements
6	clear()	Removes all elements
7	size()	Returns number of elements
8	isEmpty()	Checks if collection is empty
9	contains()	Checks if element exists
10	containsAll()	Checks if all elements exist
11	iterator()	Returns iterator object
12	toArray()	Converts to array
13	toArray(T[] a)	Converts to specific type array
14	stream()	Returns sequential stream
15	parallelStream()	Returns parallel stream

Example –

```
package collections;

import java.util.ArrayList;
import java.util.List;

public class nongeneric_collection {

    public static void main(String[] args) {
        List students_data=new ArrayList();

        students_data.add(50);
        students_data.add("Sagar");
        students_data.add(9.45);
        students_data.add('A');

        students_data.add(1, 4091764583501);
        students_data.addFirst("Pune");
        students_data.addLast("Satara");
        System.out.println("contains: "+students_data.contains("Pun"));
        //System.out.println("clear: "+students_data.clear());
        System.out.println("get: "+students_data.get(2));
        System.out.println("indexof: "+students_data.indexOf(50));
        System.out.println("getfirst: "+students_data.getFirst());
        System.out.println("getlast: "+students_data.getLast());
        System.out.println("isempty: "+students_data.isEmpty());
        System.out.println("lastindexof: "+students_data.lastIndexOf('A'));
        System.out.println("remove_by_index: "+students_data.remove(1));
        System.out.println("remove_by_objects: "+students_data.remove(5));
        System.out.println("removefirst: "+students_data.removeFirst());
        System.out.println("removelast: "+students_data.removeLast());
        System.out.println("set: "+students_data.set(1, "India"));
        System.out.println("size: "+students_data.size());
        //System.out.println("sort: "+students_data.sort(null));
        System.out.println(students_data);
    }
}
```

Output –

```
contains: false
get: 409176458350
indexof: 1
getfirst: Pune
getlast: Satara
```

isempty: false
lastindexof: 5
remove_by_index: 50
remove_by_objects: Satara
removefirst: Pune
removelast: A
set: Sagar
size: 3
[409176458350, India, 9.45]